lecture_29

October 9, 2026

1 Lecture 29: Genetic Algorithm - Benchmarking

`{note} Lecture 28 ran the Genetic Algorithm on the Ackley function, using real-valued operators. This lecture moves to TSPLIB, the same way Lecture 26 did for Simulated Annealing: calibrate population size and mutation rate on a small instance (`eil51`), then test whether that calibration still helps on a substantially larger one (`eil101`).`

Learning Objectives

By the end of this notebook, you will be able to: 1. Instantiate the Genetic Algorithm's operators for permutations (order crossover, swap mutation, tournament selection). 2. Calibrate population size and mutation rate on a small TSPLIB instance under a limited budget. 3. Compare how a population-based calibration scales to a larger instance against Simulated Annealing's scaling behaviour (Lecture 26).

Prerequisites: Genetic Algorithm - Algorithm (Lecture 28); Simulated Annealing - Benchmarking (Lecture 26).

Estimated time: 50 minutes

1.1 Permutation Operators

Lecture 28's real-valued operators (arithmetic crossover, Gaussian mutation) do not make sense for a tour — a valid tour must contain every stop exactly once, and averaging two tours' coordinates is not meaningful. This lecture specializes `ga()`'s pluggable operators the way Lecture 26 specialized `sa()`'s neighbourhood: for permutations.

- **Selection**: *tournament selection* — repeatedly keep the fittest of a small random group.
- **Crossover**: *order crossover (OX1)* — copy a random slice from one parent, fill the rest from the other parent in relative order, skipping stops already copied.
- **Mutation**: *swap mutation* — with probability ϵ, exchange two randomly chosen positions.

These are exactly the operators the Chennai-based version of this lecture used before this module's restructuring — now meeting a real TSPLIB benchmark instead.

[1]:
```
import random
import numpy as np
```

```
import matplotlib.pyplot as plt

def ga(S_o, f, SM, u, CM, l, p, MM, e, NM, n=500, t=1e-8):
    """Genetic Algorithm.

    Parameters
    ----------
    S_o : initial population (list of solutions)
    f   : objective function, f(s) -> float (minimized)
    SM  : selection method, SM(S, u, f) -> u parents
    CM  : crossover method, CM(parents, l, p) -> l offspring, p parents per␣
 ↪offspring
    MM  : mutation method, MM(offspring, e) -> mutated offspring (probability e␣
 ↪each)
    NM  : natural selection, NM(S, m, f) -> next generation of size m
    n   : maximum number of generations
    t   : convergence threshold on improvement in the best solution
    """
    S = S_o
    s_b = min(S, key=f)                 # best solution found so far
    F_b = [f(s_b)]
    k, r, converged = 0, float("inf"), False
    while not converged:
        S_p = SM(S, u, f)                           # select parents
        S_c = CM(S_p, l, p)                         # crossover -> offspring
        S_c = MM(S_c, e)                            # mutate offspring
        S = NM(S + S_c, len(S_o), f)                 # replace: parents + offspring␣
 ↪-> next population
        s_p = min(S, key=f)
        if f(s_p) < f(s_b):
            r = f(s_b) - f(s_p)
            s_b = s_p
        F_b.append(f(s_b))
        k += 1
        if k >= n or r <= t:
            converged = True
    return s_b, F_b
```

[2]:
```
def SM_tournament(S, u, f, k=3):
    """Select `u` parents by repeated tournament selection.

    Parameters
    ----------
    S : sequence
        Current population of candidate tours.
    u : int
        Number of parents to return.
```

```
    f : callable
        Objective function (lower is better).
    k : int, default=3
        Tournament size used at each selection step.

    Returns
    -------
    list
        A list of `u` parent tours, each chosen as the fittest of `k`
        randomly sampled candidates.
    """
    return [min(random.sample(S, k), key=f) for _ in range(u)]


def CM_ox1(S_p, l, p):
    """Create offspring tours using order crossover (OX1).

    Parameters
    ----------
    S_p : sequence
        Parent population.
    l : int
        Number of offspring to generate.
    p : int
        Number of parents used per offspring.

    Returns
    -------
    list
        A list of offspring tours (each a tuple of stops).
    """
    S_c = []
    for _ in range(l):
        # Sample two parents for the current offspring.
        a, b = random.sample(S_p, p)

        n = len(a)
        i, j = sorted(random.sample(range(n), 2))

        # Step 1: Copy the selected slice from parent `a`.
        child = [None] * n
        child[i:j + 1] = a[i:j + 1]

        # Step 2: Fill the remaining positions in the order they appear in
        # parent `b`, skipping any city already copied.
        fill = [g for g in b if g not in child[i:j + 1]]
        k = 0
```

```
        for idx in range(n):
            if child[idx] is None:
                child[idx] = fill[k]
                k += 1

        S_c.append(tuple(child))
    return S_c


def MM_swap(S_c, e):
    """Apply swap mutation with probability `e` to each offspring.

    Parameters
    ----------
    S_c : sequence
        Candidate offspring tours.
    e : float
        Mutation probability in [0, 1].

    Returns
    -------
    list
        Mutated offspring tours.
    """
    out = []
    for s in S_c:
        s = list(s)
        if random.random() < e:
            i, j = random.sample(range(len(s)), 2)
            s[i], s[j] = s[j], s[i]
        out.append(tuple(s))
    return out


def NM_elitist(S, m, f):
    """Keep the `m` fittest solutions from the combined parent/offspring set.

    Parameters
    ----------
    S : sequence
        Combined population.
    m : int
        Target population size.
    f : callable
        Objective function (lower is better).

    Returns
```

```
    -------
    list
        The best `m` solutions.
    """
    return sorted(S, key=f)[:m]


def TSP(s):
    """Return the tour length for a permutation `s` of customer indices.

    This function assumes a global distance matrix `D` is defined, with
    `D[i, j]` giving the distance between city `i` and city `j`. The tour is
    interpreted as: depot -> s[0] -> ... -> s[-1] -> depot.

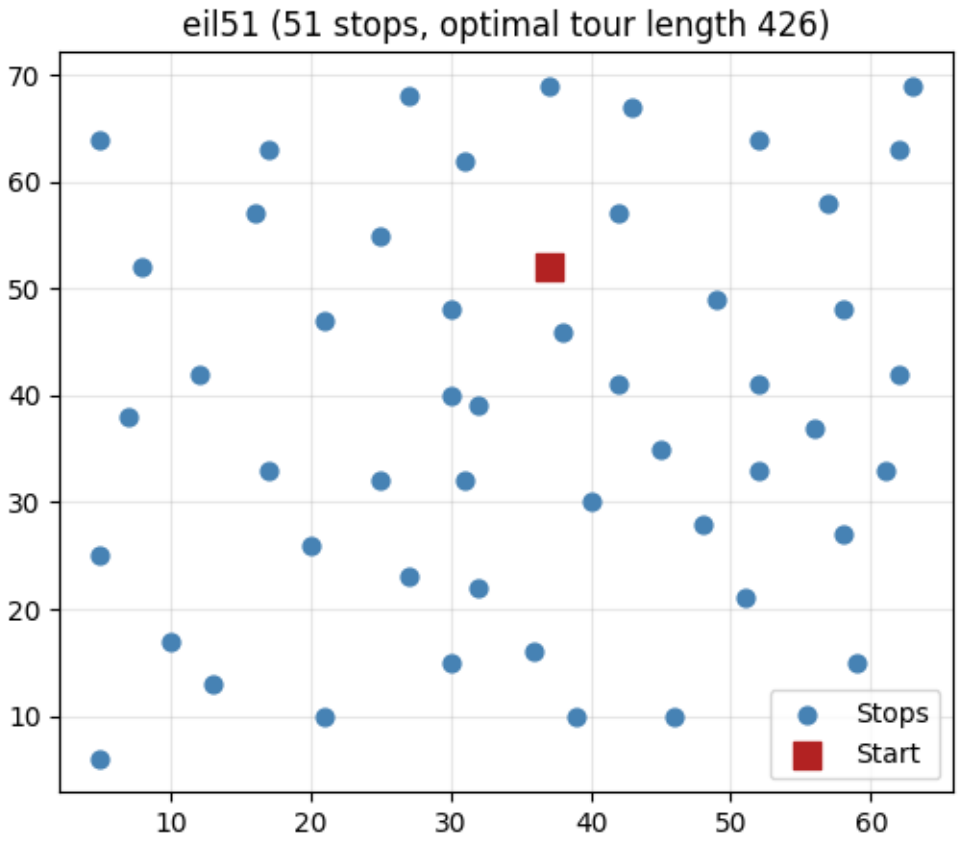
    Parameters
    ----------
    s : sequence
        A tour represented as a permutation of customer indices.

    Returns
    -------
    float
        Total travel cost of the tour.
    """
    c = D[0, s[0]]
    for i in range(len(s) - 1):
        c += D[s[i], s[i + 1]]
    c += D[s[-1], 0]
    return c
```

[4]:
```
# eil51: 51-city problem
coords = [
    (37, 52), (49, 49), (52, 64), (20, 26), (40, 30), (21, 47), (17, 63), (31,␣
 ↪62), (52, 33), (51, 21),
    (42, 41), (31, 32), (5, 25), (12, 42), (36, 16), (52, 41), (27, 23), (17,␣
 ↪33), (13, 13), (57, 58),
    (62, 42), (42, 57), (16, 57), (8, 52), (7, 38), (27, 68), (30, 48), (43,␣
 ↪67), (58, 48), (58, 27),
    (37, 69), (38, 46), (46, 10), (61, 33), (62, 63), (63, 69), (32, 22), (45,␣
 ↪35), (59, 15), (5, 6),
    (10, 17), (21, 10), (5, 64), (30, 15), (39, 10), (32, 39), (25, 32), (25,␣
 ↪55), (48, 28), (56, 37),
    (30, 40),
]
depot = 0
customers = list(range(1, len(coords)))
optimal = 426
```

```
n = len(coords)
X = np.array([c[0] for c in coords], dtype=float)
Y = np.array([c[1] for c in coords], dtype=float)
D = np.zeros((n, n))
for i in range(n):
    for j in range(n):
        D[i, j] = np.hypot(X[i] - X[j], Y[i] - Y[j])

fig, ax = plt.subplots(figsize=(6, 5))
ax.scatter(X[1:], Y[1:], c="steelblue", s=40, zorder=2, label="Stops")
ax.scatter(X[0], Y[0], c="firebrick", marker="s", s=100, zorder=3,␣
 ↪label="Start")
ax.set_title("eil51 (51 stops, optimal tour length 426)")
ax.legend()
ax.grid(alpha=0.3)
plt.show()
```



1.2 Calibrating Population Size and Mutation Rate

Under a tight budget (100 generations), averaged over 10 seeds per configuration.

[6]:
```
def run_ga(mu, eps, n_gens=100, n_seeds=10, D=D, customers=customers):
    results = []
    for seed in range(1, n_seeds + 1):
        random.seed(seed)
        S_o = [tuple(random.sample(customers, len(customers))) for _ in␣
 ↪range(mu)]
        s_b, F_b = ga(S_o, lambda s: TSP(s), SM_tournament, mu, CM_ox1, mu, 2,␣
 ↪MM_swap, eps, NM_elitist,
                      n=n_gens, t=1e-9)
        results.append(TSP(s_b))
    return np.array(results)

mu_results = {}
for mu in [10, 30, 60]:
    vals = run_ga(mu, eps=0.2)
    mu_results[mu] = vals
    gap = (vals.mean() - optimal) / optimal * 100
    print(f"mu={mu:<4} -> mean cost = {vals.mean():.2f}   gap = {gap:.1f}%")

print()
eps_results = {}
for eps in [0.05, 0.2, 0.5]:
    vals = run_ga(30, eps)
    eps_results[eps] = vals
    gap = (vals.mean() - optimal) / optimal * 100
    print(f"eps={eps:<5} -> mean cost = {vals.mean():.2f}   gap = {gap:.1f}%")
```

```
mu=10   -> mean cost = 955.61   gap = 124.3%
mu=30   -> mean cost = 801.86   gap = 88.2%
mu=60   -> mean cost = 698.88   gap = 64.1%

eps=0.05  -> mean cost = 924.84   gap = 117.1%
eps=0.2   -> mean cost = 801.86   gap = 88.2%
eps=0.5   -> mean cost = 756.03   gap = 77.5%
```

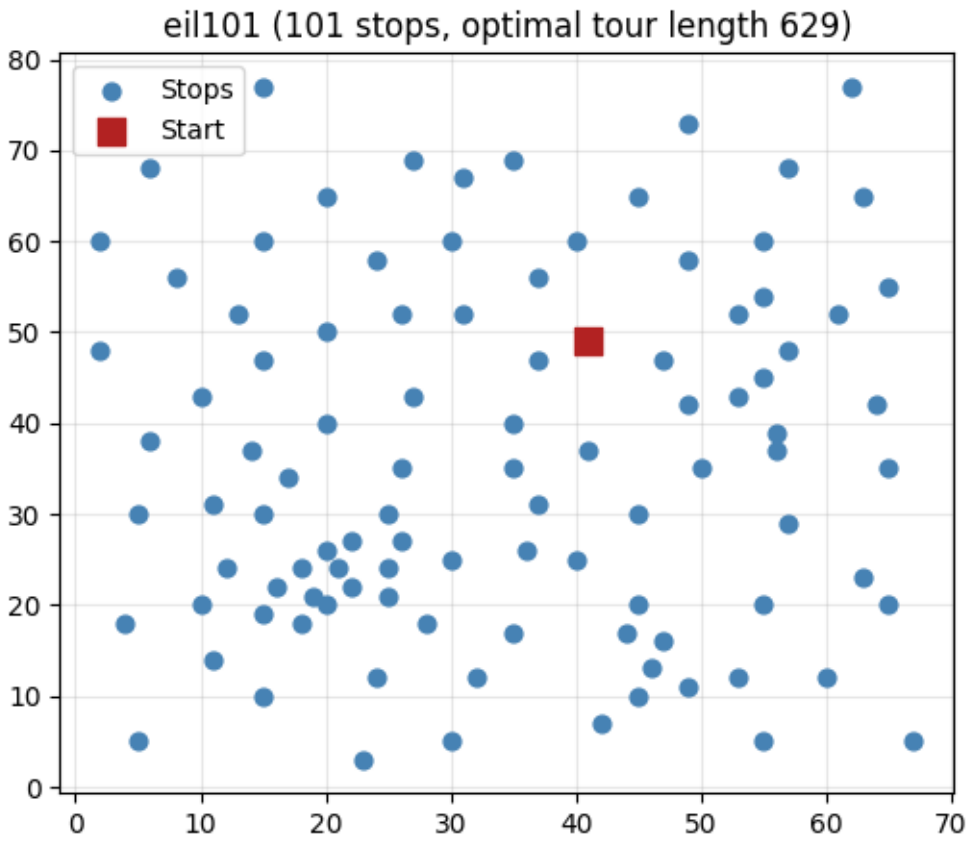
{note} Both parameters show a clean monotonic pattern: larger μ and higher ϵ both help, consistently, across the range tested - more diversity (from a bigger population or from more frequent mutation) keeps paying off rather than trading off against itself, unlike Simulated Annealing's cooling rate (Lecture 26), which had a genuine interior sweet spot. Combine both: $\mu=60$, $\epsilon=0.5$.

1.3 Benchmarking at Scale

[7]:
```
# eil101: 101-city problem
coords = [
    (41, 49), (35, 17), (55, 45), (55, 20), (15, 30), (25, 30), (20, 50), (10,␣
 ↪43), (55, 60), (30, 60),
    (20, 65), (50, 35), (30, 25), (15, 10), (30, 5), (10, 20), (5, 30), (20,␣
 ↪40), (15, 60), (45, 65),
    (45, 20), (45, 10), (55, 5), (65, 35), (65, 20), (45, 30), (35, 40), (41,␣
 ↪37), (64, 42), (40, 60),
    (31, 52), (35, 69), (53, 52), (65, 55), (63, 65), (2, 60), (20, 20), (5,␣
 ↪5), (60, 12), (40, 25),
    (42, 7), (24, 12), (23, 3), (11, 14), (6, 38), (2, 48), (8, 56), (13, 52),␣
 ↪(6, 68), (47, 47),
    (49, 58), (27, 43), (37, 31), (57, 29), (63, 23), (53, 12), (32, 12), (36,␣
 ↪26), (21, 24), (17, 34),
    (12, 24), (24, 58), (27, 69), (15, 77), (62, 77), (49, 73), (67, 5), (56,␣
 ↪39), (37, 47), (37, 56),
    (57, 68), (47, 16), (44, 17), (46, 13), (49, 11), (49, 42), (53, 43), (61,␣
 ↪52), (57, 48), (56, 37),
    (55, 54), (15, 47), (14, 37), (11, 31), (16, 22), (4, 18), (28, 18), (26,␣
 ↪52), (26, 35), (31, 67),
    (15, 19), (22, 22), (18, 24), (26, 27), (25, 24), (22, 27), (25, 21), (19,␣
 ↪21), (20, 26), (18, 18),
    (35, 35),
]
depot = 0
customers = list(range(1, len(coords)))
optimal = 629


n = len(coords)
X = np.array([c[0] for c in coords], dtype=float)
Y = np.array([c[1] for c in coords], dtype=float)
D = np.zeros((n, n))
for i in range(n):
    for j in range(n):
        D[i, j] = np.hypot(X[i] - X[j], Y[i] - Y[j])

fig, ax = plt.subplots(figsize=(6, 5))
ax.scatter(X[1:], Y[1:], c="steelblue", s=40, zorder=2, label="Stops")
ax.scatter(X[0], Y[0], c="firebrick", marker="s", s=100, zorder=3,␣
 ↪label="Start")
ax.set_title("eil101 (101 stops, optimal tour length 629)")
ax.legend()
ax.grid(alpha=0.3)
plt.show()
```



[8]:
```
random.seed(42)
S_o = [tuple(random.sample(customers, len(customers))) for _ in range(60)]
s_b, F_b = ga(S_o, lambda s: TSP(s), SM_tournament, 60, CM_ox1, 60, 2, MM_swap,␣
 ↪0.5, NM_elitist,
              n=500, t=1e-9)
cost = TSP(s_b)
print(f"Calibrated (mu=60, eps=0.5), same budget n=500: cost = {cost:.2f}   "
      f"gap = {(cost - optimal) / optimal * 100:.1f}%")
```

```
Calibrated (mu=60, eps=0.5), same budget n=500: cost = 1025.04   gap = 63.0%
```

Worse than on `eil51`, as expected — a 101-stop tour is a much larger search space. Unlike Simulated Annealing's cooling rate, which plateaued almost immediately once its budget ran out, check whether GA keeps making progress if simply given more budget — either more generations, or a larger population.

[9]:
```
for mu, gens in [(60, 500), (60, 1000), (120, 1000)]:
    random.seed(42)
    S_o = [tuple(random.sample(customers, len(customers))) for _ in range(mu)]
    s_b, F_b = ga(S_o, lambda s: TSP(s), SM_tournament, mu, CM_ox1, mu, 2,␣
 ↪MM_swap, 0.5, NM_elitist,
```

```
                n=gens, t=1e-9)
    cost = TSP(s_b)
    print(f"mu={mu:<4} gens={gens:<5} -> cost = {cost:.2f}   gap = {(cost -␣
↪optimal) / optimal * 100:.1f}%")
```

```
mu=60   gens=500   -> cost = 1025.04   gap = 63.0%
mu=60   gens=1000  -> cost = 958.85   gap = 52.4%
mu=120  gens=1000  -> cost = 896.39   gap = 42.5%
```

Interpretation. Unlike Simulated Annealing (Lecture 26), which was completely stuck at its calibrated cooling rate until the rate itself was slowed down, the Genetic Algorithm keeps improving gradually just from more budget — more generations helps a little, a larger population on top of that helps a little more — without needing to change μ or ϵ's *character* the way r had to change character (fast → slow). This is not a coincidence: population size and mutation rate both already showed a "more is better, monotonically" pattern on `eil51`, so scaling them further (or just giving them more generations to work with) extends the same trend rather than requiring a qualitatively different setting.

> **Managerial insight**: not every algorithm's parameters fail the same way at scale. Simulated Annealing's single search thread needed its schedule explicitly re-tuned before more time helped at all; the Genetic Algorithm's population-based search degraded more gracefully, absorbing extra budget productively even without retuning. Knowing *which failure mode* your algorithm has is itself something worth calibrating for, not just the parameter values.

1.4 Take-Away Exercises

1.4.1 Exercise 1 — Population Size vs. Generations

Holding total function evaluations roughly constant ($\mu \times$ generations), compare $\mu = 60$/1000 generations against $\mu = 120$/500 generations on `eil101`. Does it matter which one you scale up?

1.4.2 Exercise 2 — A Larger Mutation Rate Still

This lecture's calibration on `eil51` found $\epsilon = 0.5$ better than 0.2 and 0.05. Test $\epsilon = 0.8$ on both `eil51` and `eil101` — does the "more mutation helps" trend continue, or does it eventually reverse?

1.5 Circling Back

- **Lecture 28 (Genetic Algorithm: Algorithm)**: the `ga()` engine above is unchanged; only the operators (`SM_tournament`, `CM_ox1`, `MM_swap`) differ from Lecture 28's real-valued ones, the same engine/operator split Lecture 27 introduced.
- **Lecture 26 (Simulated Annealing: Benchmarking)**: both lectures tested scale transfer on the same `eil51`/`eil101` pair; the two algorithms scaled differently, a genuine finding worth carrying into Lecture 32's test of Ant Colony Optimization.

1.6 Moving Forward

- **Lecture 30 (Ant Colony Optimization: Motivation & Pseudocode)**: introduces Lecture 23's third paradigm, swarm intelligence.

1.7 Further Reading

- Reinelt, G. (1991). "TSPLIB — A Traveling Salesman Problem Library." *ORSA Journal on Computing*, 3(4), 376-384.
- Larrañaga, P., Kuijpers, C.M.H., Murga, R.H., Inza, I., and Dizdarevic, S. (1999). "Genetic Algorithms for the Travelling Salesman Problem: A Review of Representations and Operators." *Artificial Intelligence Review*, 13(2), 129-170.